Scuola universitaria professionale della Svizzera italiana
Dipartimento tecnologie innovative

SUPSI

2D Image Editor

Students

Mattia Cainarca
Luca Fantò
Antonio Marroffino

Degree Course	Modules	Accademic Year
Bachelor Ingegneria Informatica	Software Engineering II - Operating Systems	2024 - 2025

Stakeholders	Date
Giancarlo Corti Amos Brocco Matteo Besenzoni	16/12/2024

Index

Context and Motivation

Project goals

The project focuses on **improving** the **skills** and **soft skills** involved in developing a software project in teams:

- Configuration management (source code versioning, product versioning, software dependency, build and distribution of the final software product)
- Requirements elicitation and management
- Software design and development
- Internationalization
- Unit tests and End-to-End software testing
- User interaction

Requirements Elicitation and Analysis

User requirements

Load and **display PNM images** from the filesystem

Displays contextual **feedback to** guide **user**

Save modified images in **supported PNM formats**

Create and **execute transformation** pipelines

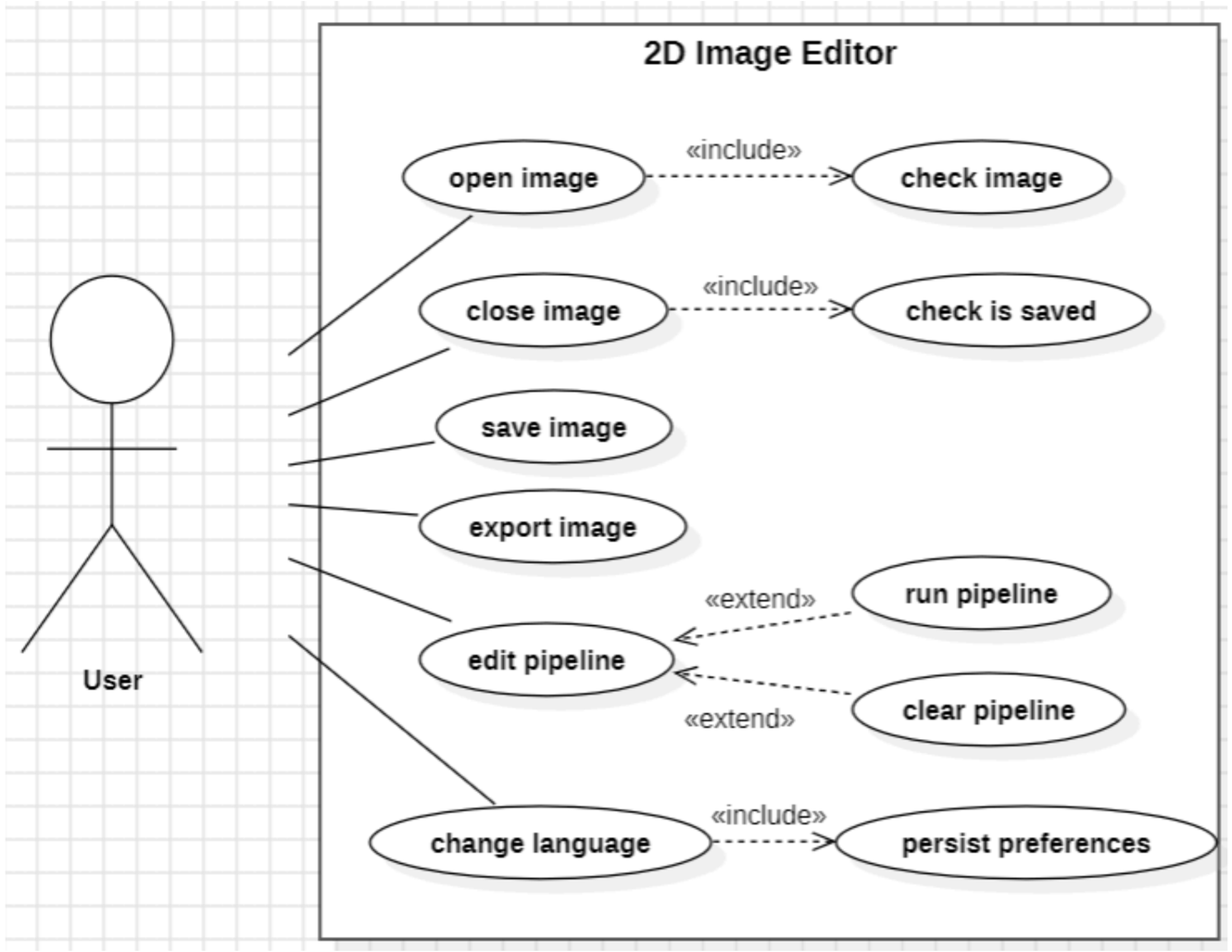
Supports **different languages** in the UI

Display detailed **application information**

Standalone application with a graphical user interface

Problem

Use case diagram



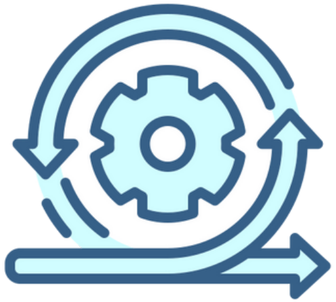
Approach

AGILE

The project was managed following the **Agile methodology**, which allowed us to **adapt quickly to changes** that emerged during development. We organized the work in **short iterations**, ensuring that **functional increments were delivered** at the end of each cycle.

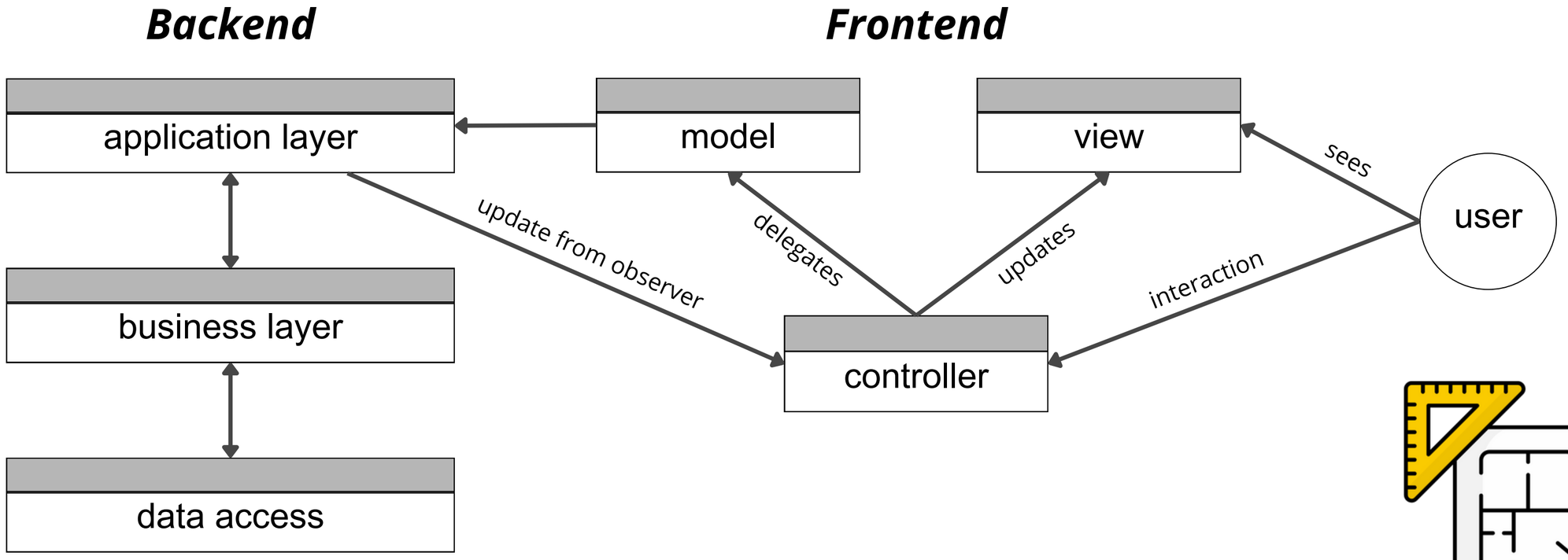
User requirements were collected through brainstorming and periodic feedback, and **inserted into a project backlog**.

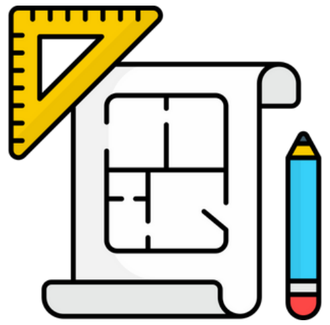
This approach allowed for **clear functionality traceability** and **close collaboration** between development teams and stakeholders.



Approach

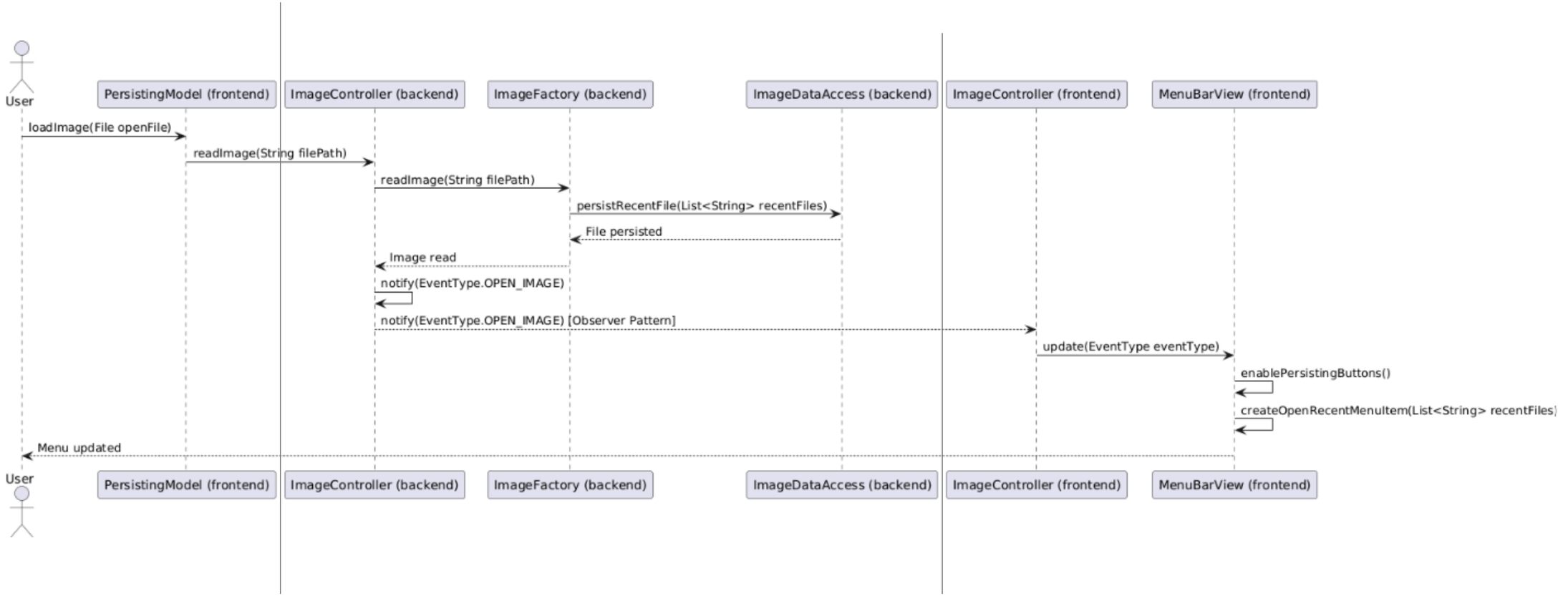
Architectural design: Layering - MVC



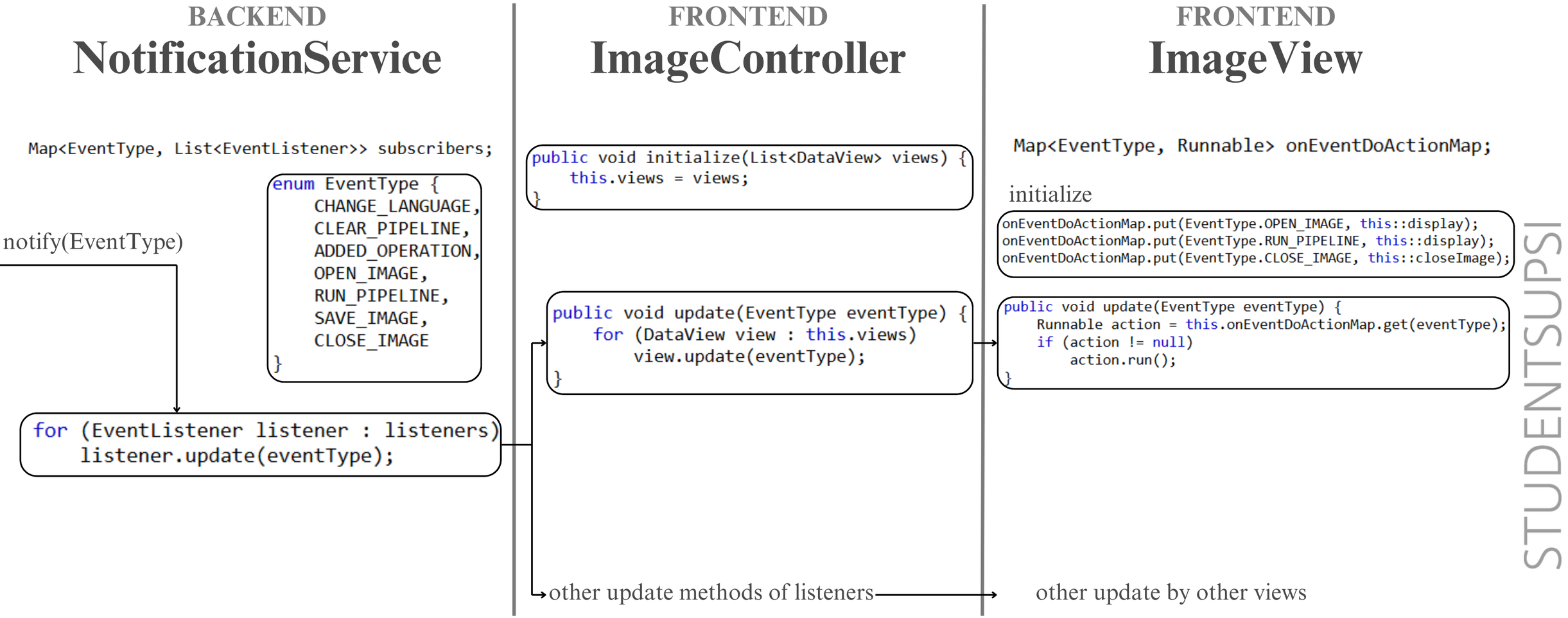


Approach

Open Recent



Approach: Observer pattern

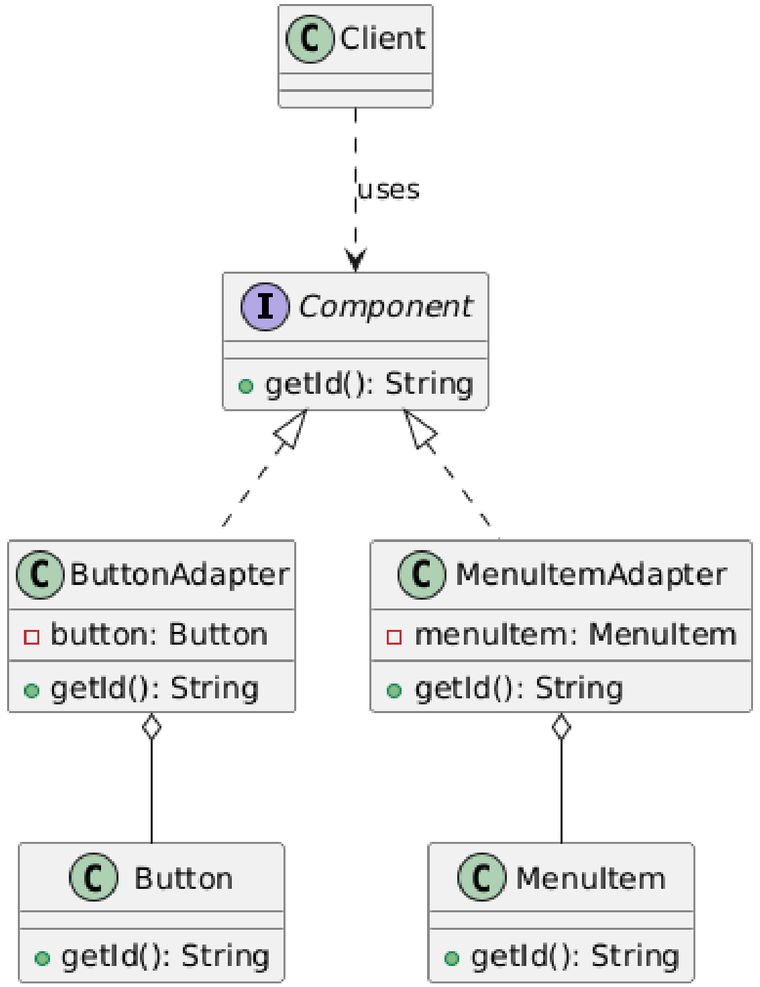


Approach

Adapter Pattern

The **Adapter Pattern** allows handling **different objects** (Button and MenuItem) **as Component**, improving **maintainability**.

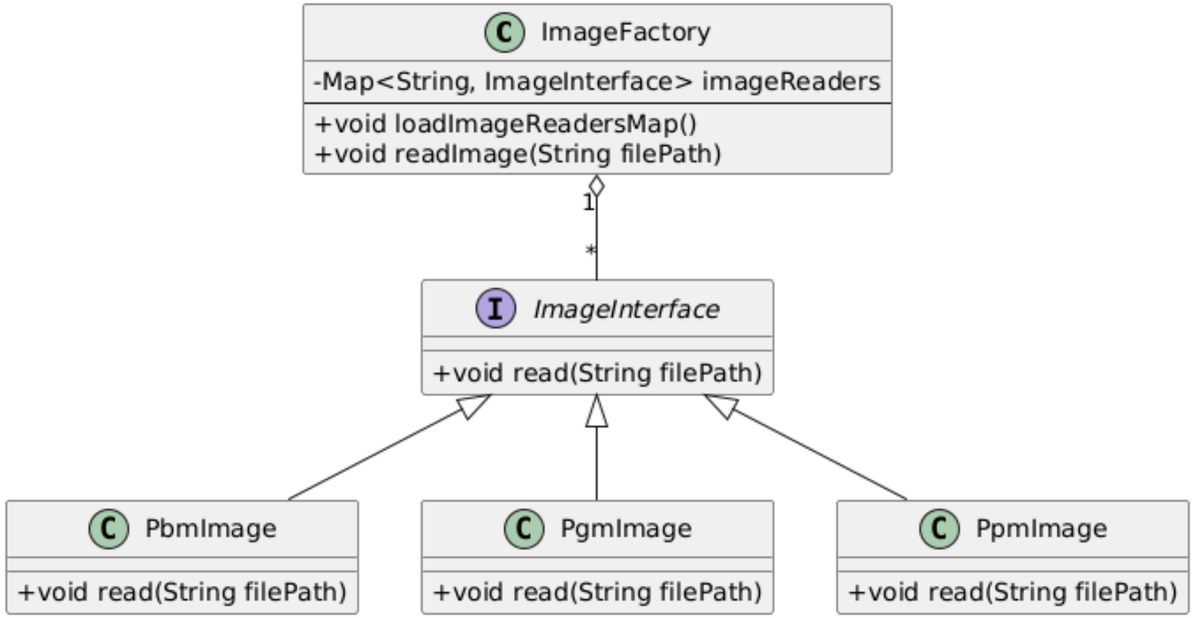
It also **enables** future **extensibility** by adding new adapters without changing the client code.



Approach

Abstract Factory Pattern

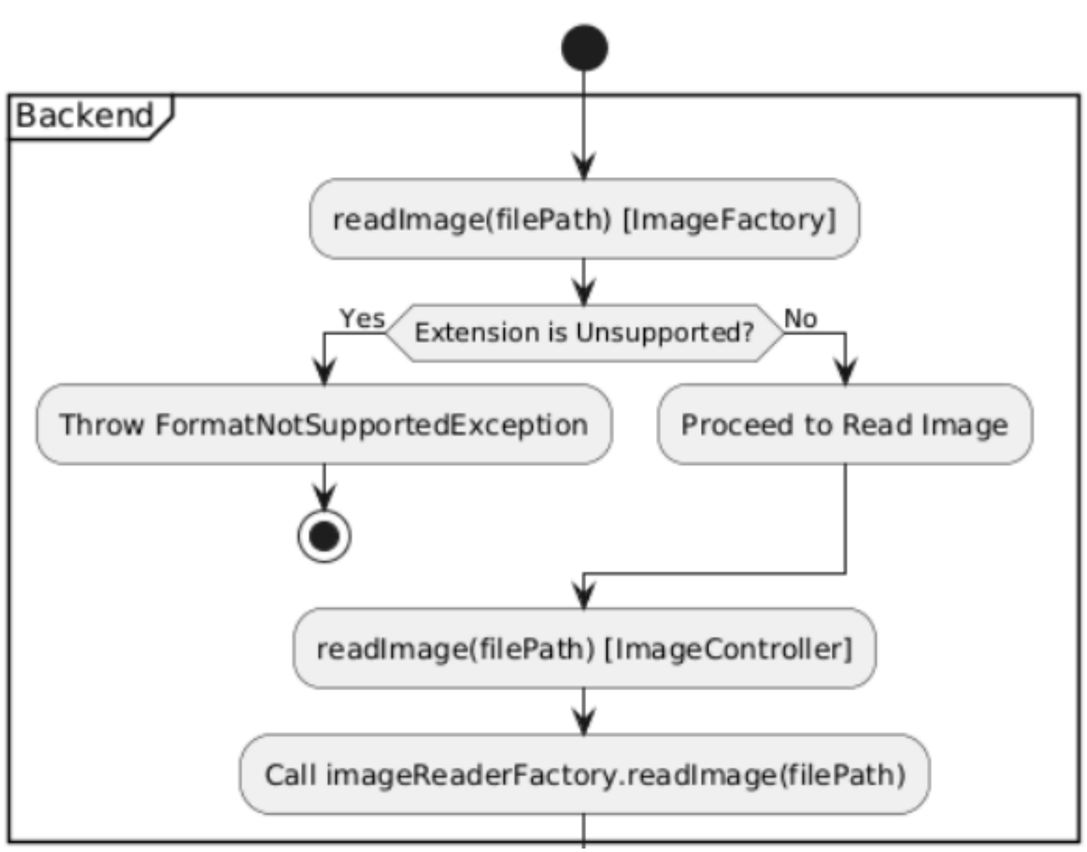
ImageFactory class **enables** the **creation and management** of different types of image readers **dynamically** based on their file extensions.

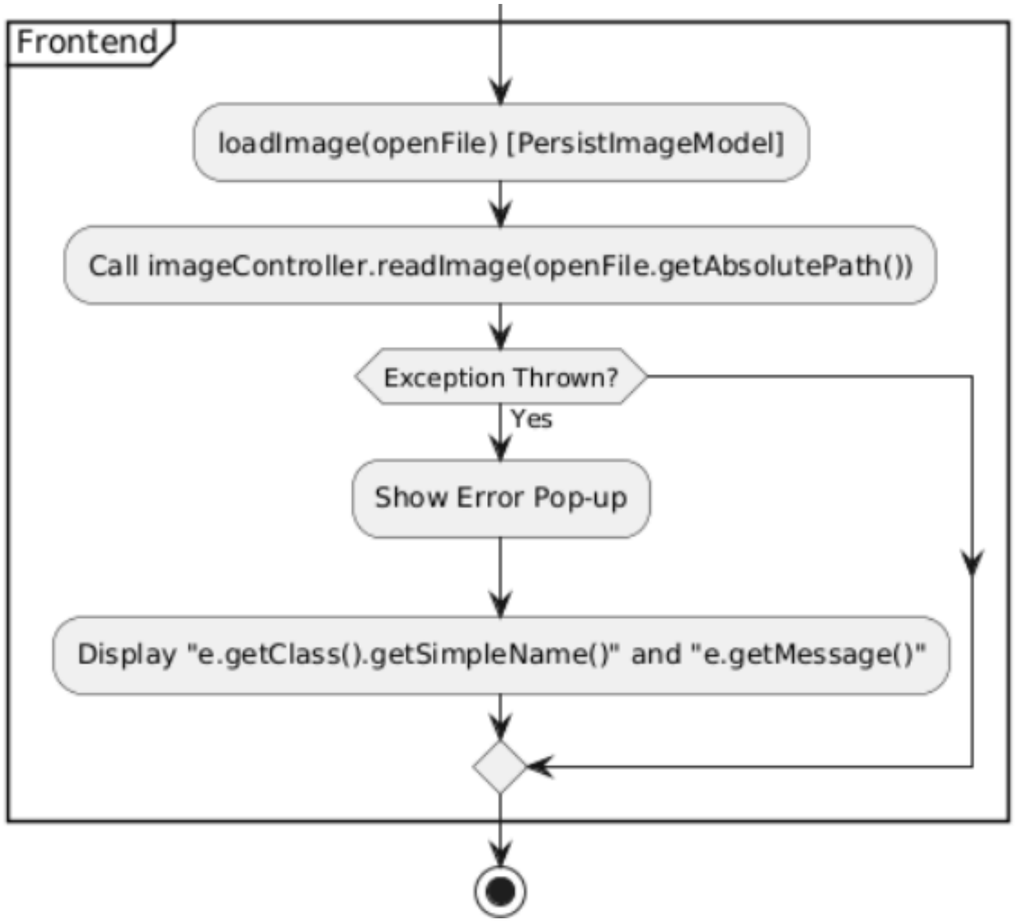



```
public void readImage(String filePath) throws ... {
        String extension = this.getFileExtension(filePath).toUpperCase();
        this.currentImageReader = this.imageReaders.get(extension);
        if (this.currentImageReader == null)
            throw new FormatNotSupportedException("...");
        this.currentImageReader.read(filePath);
        this.currentImage = this.currentImageReader.getImage();
        this.persistRecentFile(filePath);
}
```

Exceptions Management

Flow of FormatNotSupportedException





Test

Unit tests

We ensured the **reliability** and **functionality** of the 2D editor through a rigorous testing process using **JUnit 5** and **Mockito**.

Used to **structure** and **automate unit tests**, ensuring the correct execution

Used to **mock external dependencies**, isolating code units during tests.

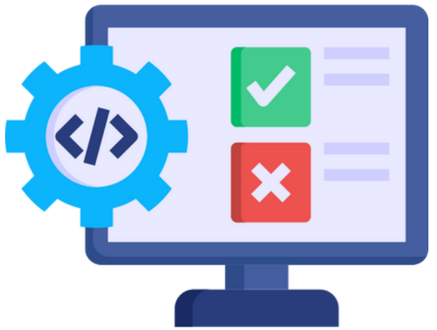
Code coverage tool for Java, providing detailed reports to track which parts of the code are tested and which are not.

Test

User Interface tests

To verify that the **graphical interface** responds correctly to the **expected functionalities after** a given **event** occurs, such as clicking on a button, **end-to-end tests were necessary**.

Tests are **carried out by** a kind of **robot**, provided by the JUnit dependency, which **through** the ***fx:id*** elements in the scene **allows to click on them** and perform the actions related to them.



Results

Achieved results in line with established requirements

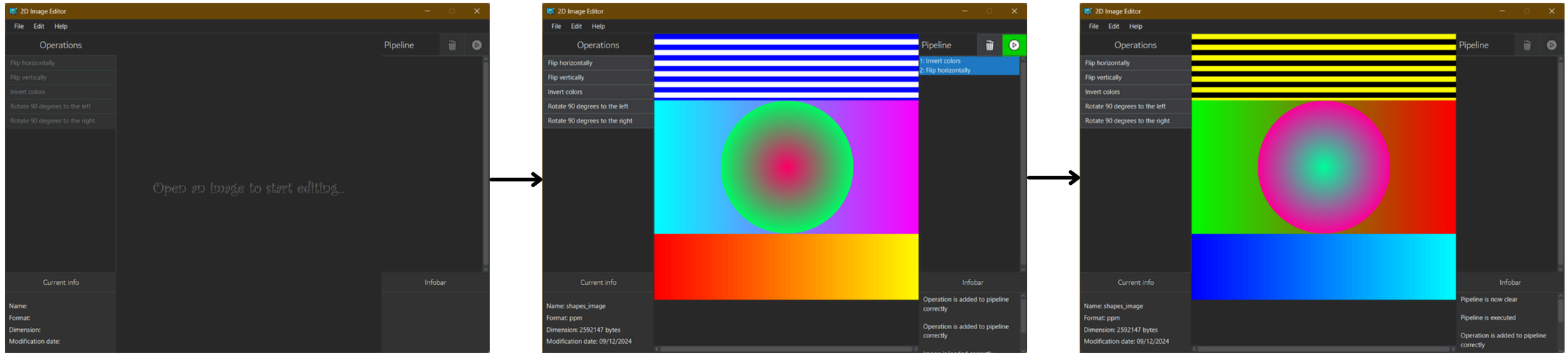
The **final software product** meets the initial **requirements**, is compatible with all **operating systems**, and supports multiple **languages**.

The editor operates correctly, offering users a range of available **operations** for creating and modifying 2D images, specifically in the **PNM format**.

Additionally, the program allows **exporting** files in a format different from the original and provides the option to **open recently used files** for added convenience.

Result

Static software demo



Conclusion

Possible optimizations and improvements

To ensure **adaptability** for future needs, the program has been developed with a focus on **extensibility**. The architecture is designed to support potential future implementations seamlessly, making it straightforward to add new editing operations.

This approach aligns with the **Open-Closed Principle** (OCP), ensuring the codebase is open for extensions but closed for modifications. As a result, the program is easier to **maintain** and **expand** over time, fostering long-term **flexibility** and **scalability**.

THANK YOU FOR YOUR ATTENTION